CNSCC.160 Fundamentals of Communications Systems

Lab 1: Introduction to MATLAB

1 Introduction

The name MATLAB stands for MATrix LABoratory. MATLAB was written originally to provide easy access to matrix software developed by the LINPACK (linear system package) and EISPACK (Eigen system package) projects.

MATLAB is a numerical computing environment developed by MathWorks. MATLAB allows matrix manipulations, plotting of functions and data, and implementation of algorithms.

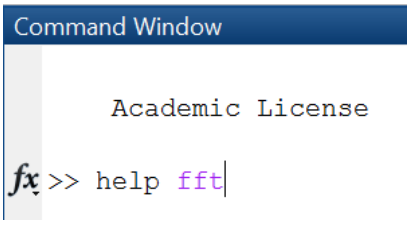
During the next few practical sessions, we will be focusing on examples for sending data over a long distance by using MATLAB as the simulation tool. In particular, we will focus on analogue modulation (AM) used for Radio and TV broadcasting; and basic digital modulation techniques used in the domestic telephone system.

This tutorial is adapted from: Introduction to MATLAB for Engineering Students by David Houcque

2 Get started

2.1 Getting help

You can get help by typing the commands, for example:



2.2 Arithmetic operators

Symbol	Operation	Example
+	Addition	2+5
-	Subtraction	9-3
*	Multiplication	2*3
/	Division	4/2
^	Power	9^3

Please refer to Table A.1 in Appendix A for more commands to manage the workshop in MATLAB.

2.3 Create MATLAB variables

MATLAB variables are created with an assignment operator "=" The syntax of variable assignment is:

variable name = a value (or an expression)

where expression is a combination of numerical values, mathematical operators, variables, and function calls. On other words, expression can involve:

- Manual entry
- Build-in functions
- User-defined functions

PS: pi refers to π in MATLAB.

```
>> x=10

x =

    10
```

```
>> y=2*7+2/pi

y =

   14.6366
```

2.4 Create MATLAB vectors/matrices

MATLAB can create and manipulate arrays of 1 (vectors), 2 (matrices), or more dimensions. For example,

Row vectors: a = [1, 2, 3, 4] is a 1x4 matrix

Column vectors: b = [5; 6; 7; 8; 9] is a 5x1 matrix

TASK 1: Output a 5x4 matrix. The elements in the matrix can be any value from 0 to 9.

2.5 Controlling the hierarchy of operations or precedence

The order in which MATLAB performs arithmetic operations is exactly that taught in high school algebra courses.

- Exponentiations
- Multiplications and divisions
- Additions and subtractions.

TASK 2: Using MATLAB to calculate $\frac{2}{3+5^2}+\frac{4}{7}\times\frac{2}{9}$

2.6 Mathematical Functions in MATLAB

MATLAB offers many predefined mathematical functions, called *built-ins*, for technical computing, e.g.,

cos(x)	Cosine	abs(x)	Absolute value
sin(x)	Sine	angle(x)	Phase angle
exp(x)	Exponential	conj(x)	Complex conjugate
sqrt(x)	Square root	log(x)	Natural logarithm

In additional to the elementary functions, MATLAB includes a number of predefined constant values, e.g.,

`pi`	The π number, $\pi = 3.14159\ldots$
`i,j`	The imaginary unit i, $\sqrt{-1}$
`Inf`	The infinity, ∞
`NaN`	Not a number

TASK 3: Using MATLAB to calculate $\sin\frac{\pi}{5}$, e^5, and $\sqrt{1000}$

2.7 Manage the workshop

The contents of the workspace persist between the executions of separate commands. Therefore, it is possible for the results of one problem to have an effect on the next one. To avoid this possibility, it is a good idea to issue a clear command at the start of each new independent calculation.

```
>> clear
```

The command clear or clear all removes all variables from the workspace. This frees up system memory. In order to display a list of the variables currently in the memory,

```
>> who
```

PS: please refer to Table A.4 in Appendix A for more commands to manage the workshop in MATLAB.

3 Basic Plotting

MATLAB has an excellent set of graphic tools. Plotting a given data set or the results of computation is possible with very few commands. You are highly encouraged to plot mathematical functions and results of analysis as often as possible. Trying to understand mathematical equations with graphics is an enjoyable and very efficient way of learning mathematics. Being able to plot mathematical functions and data freely is the most important step, and this section is written to assist you to do just that.

3.1 Creating simple plots

The MATLAB command to plot a graph is plot(x,y), e.g.,

```
>> x=[1:1:6];
>> y=[3;5;2;-2;0;1];
>> plot(x,y)
```

从1开始

间隔1

不超过这个数

x=[1:2:6]

= 1 3 5

The figure we can get is shown in Figure 1.

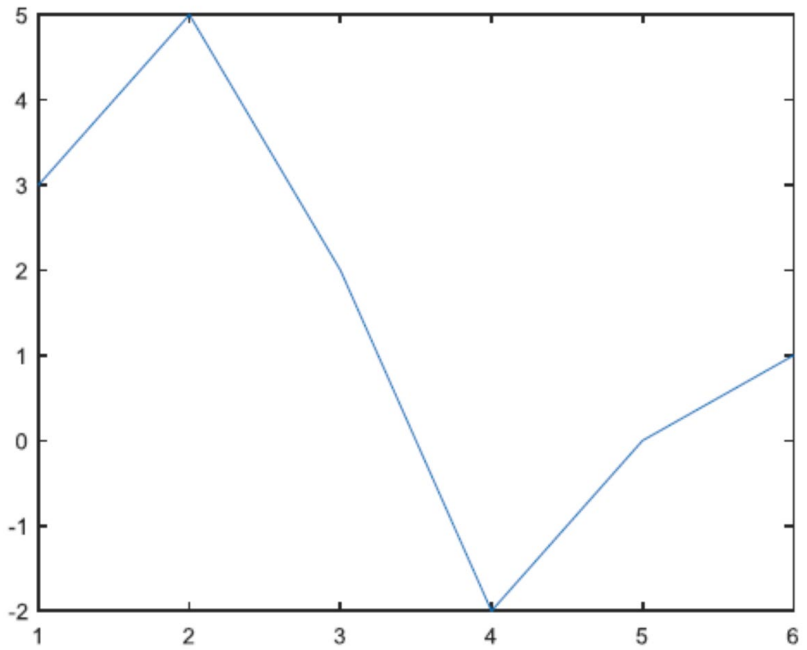


Figure 1. Figure Plot Illustration

Pay attention: DONOT use screenshot to copy the figure from MATLAB!!!

You can copy the figure from MATLAB by clicking `Edit->Copy Figure' option as shown in Figure 2. Then you can paste it into OFFICE WORD, POWERPOINT…

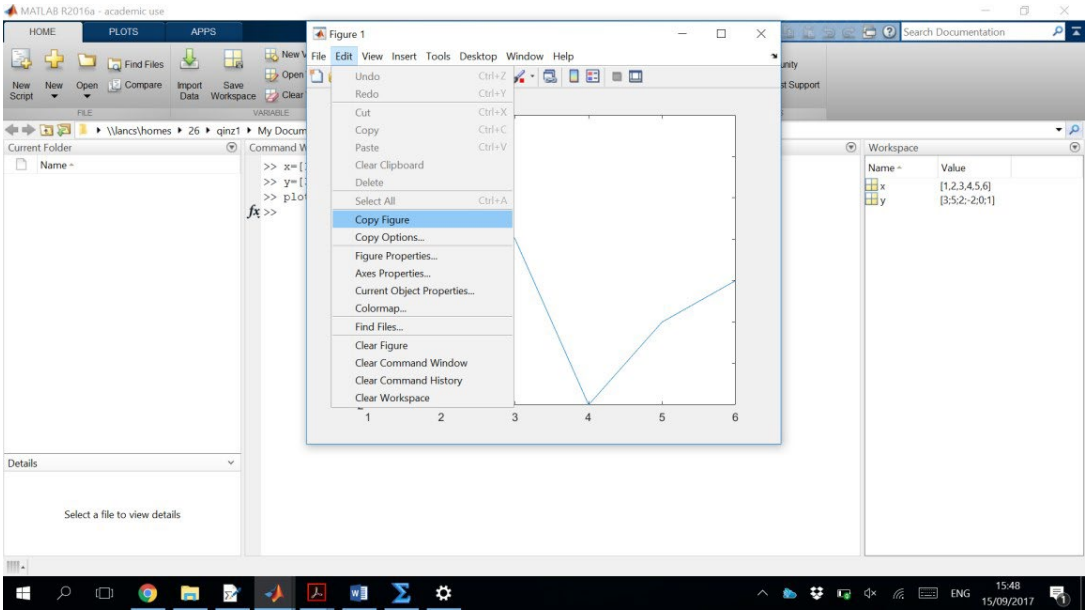


Figure 2. How to copy figure after it is plotted by MATLAB

TASK 4: Plot the function sin (x) on the interval [0,2π], with steps of $\frac{\pi}{500}$.

3.2 Adding titles, axis labels, and annotations

MATLAB enables you to add axis labels and titles. For example, using the graph from Task 4, add an x- and y-axis labels.

```
>>xlabel('x = 0:2\pi')
>>ylabel('Sine of x')
>> title('Plot the Sine wave')
```

The color of a single curve is, by default, blue, but other colors are possible. The desired color is indicated by a third argument. For example, red is selected by plot(x,y,'r'). Note the single quotes, ' ', around r. The plotted figure is as shown in Figure 3.

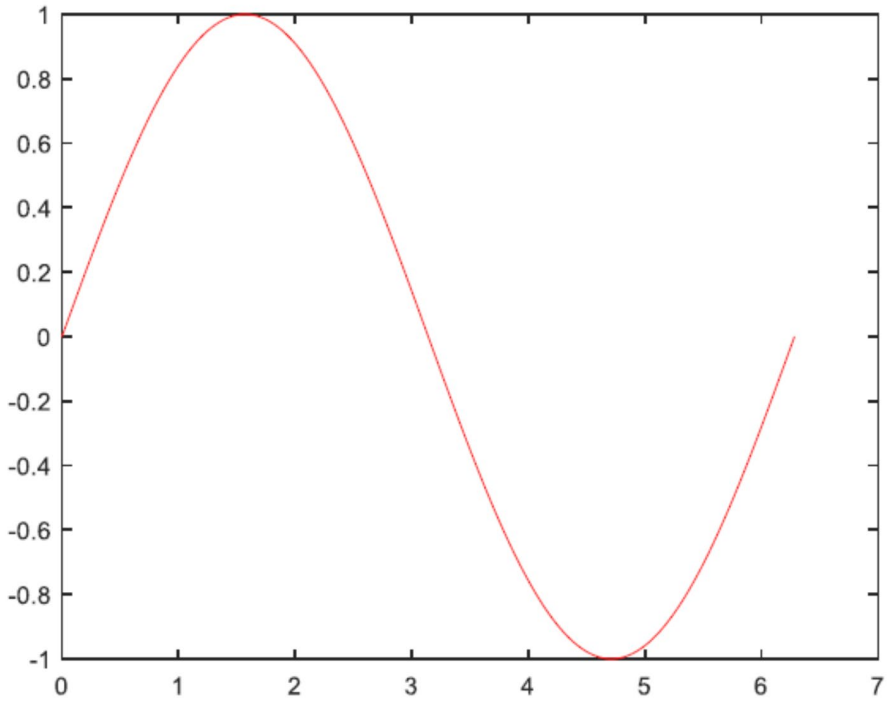


Figure 3. Plot the Sinewave in Red

3.3. Plot multiple data sets in one plot

Multiple (x,y) pairs arguments create multiple graphs with a single call to plot. For example, the following statements plot three related functions of *x in the interval [0,2π]:* $y_1 = 2\cos(x)$, $y_2 = 5\cos(2x)$, $y_3 = 0.5\cos(3x)$.

```
>> x=0:pi/500:2*pi;
>> y1 = 2*cos(x);
>> y2 = 5*cos(2*x);
>> y3 = 0.5*cos(3*x);
>> plot(x,y1,'r',x,y2,'g',x,y3,'b')
>> xlabel('0 \leq x \leq 2\pi')
>> ylabel('Cosine functions')
>> legend('2*cos(x)','5*cos(2x)','0.5*cos(3x)')
>> title('Typical example of multiple plots')
```

red green blue

The plotted figure is as shown in Figure 4.

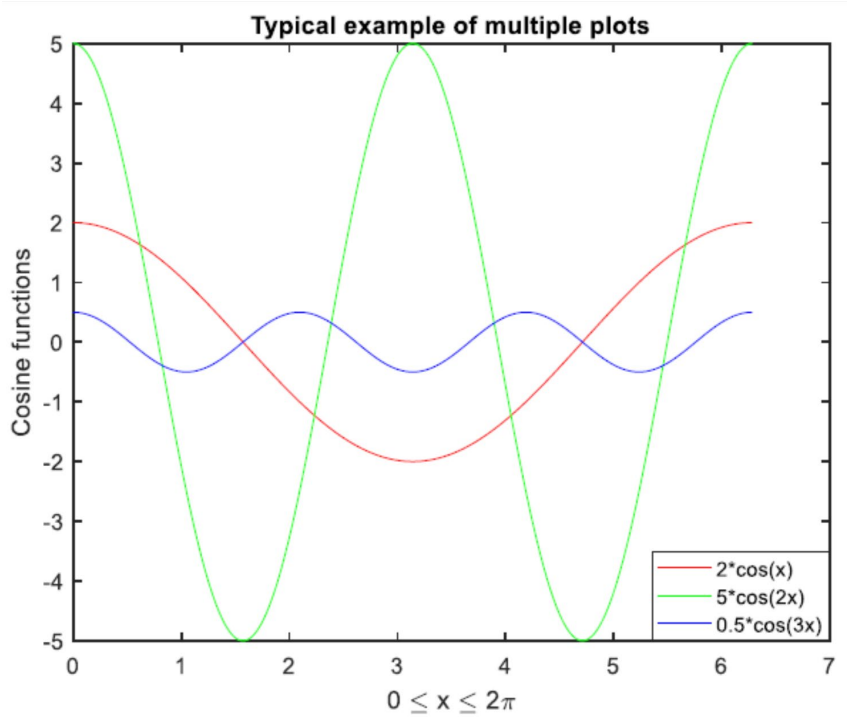


Figure 4. Typical example of multiple plots

By default, MATLAB uses line style and color to distinguish the data sets plotted in the graph. However, you can change the appearance of these graphic components or add annotations to the graph to help explain your data for presentation.

TASK 5: Figure out how to plot subfigrues in MATLAB by yourself.

3.4 Specifying line styles and markers

It is possible to specify line styles, colors, and markers (e.g., circles, plus signs, . . .) using the plot command:

plot(x,y,'style_color_marker')

The commonly used attributes for plot are listed as

Symbol	Color	Symbol	Line Style	Symbol	Marker
k	Black	−	Solid	+	Plus sign
r	Red	−−	Dashed	o	Circle
b	Blue	:	Dotted	∗	Asterisk
g	Green	−.	Dash-dot	.	Point
c	Cyan	none	No line	×	Cross
m	Magenta			s	Square
y	Yellow			d	Diamond

TASK 6: use different colors, lines, and markers to distinguish the different cosine functions shown in Figure 4.

4 Programming in MATLAB

So far in these lab sessions, all the commands were executed in the Command Window. The problem is that the commands entered in the Command Window cannot be saved and executed again for several times. Therefore, a different way of executing repeated commands with MATLAB is:

- To create a file with a list of commands,
- Save the file, and
- Run the file.

If needed, corrections or changes can be made to the commands in the file. The files that are used for this purpose are called script files or scripts for short.

4.1 Example

Consider the system of equations:

$$\begin{cases} x + 2y + 3z &= 1 \\ 3x + 3y + 4z &= 1 \\ 2x + 3y + 3z &= 2 \end{cases}$$

Find the solution x to the system of equations.

SOLUTION:

- Use the MATLAB *editor* to create a file: **File** → **New** → **M-file**.
- Enter the following statements in the file:

```
A = [1 2 3; 3 3 4; 2 3 3];
b = [1; 1; 2];
x = A\b
```

- Save the file, for example, `example1.m`.
- Run the file, in the command line, by typing:

```
>> example1
x   =
    -0.5000
     1.5000
    -0.5000
```

Note that in the above solution, the way of creating a file is for an old release. For your MATLAB that is a recent release, you may follow **Home** → **New Script** to create a file.

TASK 7: save the codes for TASK 6 in a .m file, and name it CosinePlot.m

Appendix A: Summary of Commands

Table A.1: **Arithmetic operators and special characters**

Character	Description
+	Addition
−	Subtraction
*	Multiplication (scalar and array)
/	Division (right)
^	Power or exponentiation
:	Colon; creates vectors with equally spaced elements
;	Semi-colon; suppresses display; ends row in array
,	Comma; separates array subscripts
...	Continuation of lines
%	Percent; denotes a comment; specifies output format
'	Single quote; creates string; specifies matrix transpose
=	Assignment operator
( )	Parentheses; encloses elements of arrays and input arguments
[ ]	Brackets; encloses matrix elements and output arguments

Table A.2: **Array operators**

Character	Description
.*	Array multiplication
./	Array (right) division
.^	Array power
.\	Array (left) division
.'	Array (nonconjugated) transpose

Table A.3: **Relational and logical operators**

Character	Description
$<$	Less than
$\leq$	Less than or equal to
$>$	Greater than
$\geq$	Greater than or equal to
$==$	Equal to
$\sim=$	Not equal to
&	Logical or element-wise AND
|	Logical or element-wise OR
&&	Short-circuit AND
||	Short-circuit OR

Table A.4: **Managing workspace and file commands**

Command	Description
`cd`	Change current directory
`clc`	Clear the Command Window
`clear (all)`	Removes all variables from the workspace
`clear x`	Remove x from the workspace
`copyfile`	Copy file or directory
`delete`	Delete files
`dir`	Display directory listing
`exist`	Check if variables or functions are defined
`help`	Display help for MATLAB functions
`lookfor`	Search for specified word in all help entries
`mkdir`	Make new directory
`movefile`	Move file or directory
`pwd`	Identify current directory
`rmdir`	Remove directory
`type`	Display contents of file
`what`	List MATLAB files in current directory
`which`	Locate functions and files
`who`	Display variables currently in the workspace
`whos`	Display information on variables in the workspace

CNSCC.160 Fundamentals of Communications Systems

Lab 1: Introduction to Simulink

INTRODUCTION:

Simulink is an add-on to MATLAB created by MathWorks Inc. It works with MATLAB to provide a wide range of building blocks for creating models of systems, including sources, outputs, and signal processing components. You can also create custom blocks by combining existing ones or by writing new blocks using MATLAB code or C programming.

To simulate a system in Simulink, there are two main steps. First, you create a block diagram that visually represents the system and how it changes over time. Then, you tell Simulink to run the simulation from a chosen start time to an end time.

This tutorial will introduce the key features of Simulink that will help you simulate both analogue and digital communication systems. This will allow you to examine signal waveforms and their frequency content at different points in the system.

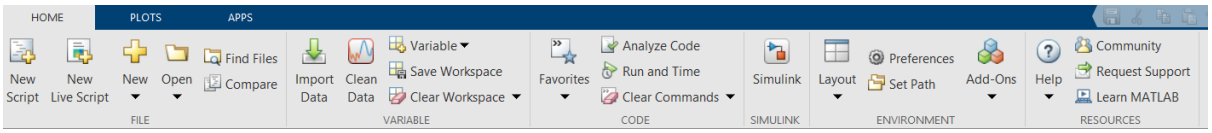
Since Simulink is closely integrated with MATLAB, you can easily switch between the two, taking full advantage of the features each offers for analysis.

GETTING STARTED IN SIMULINK

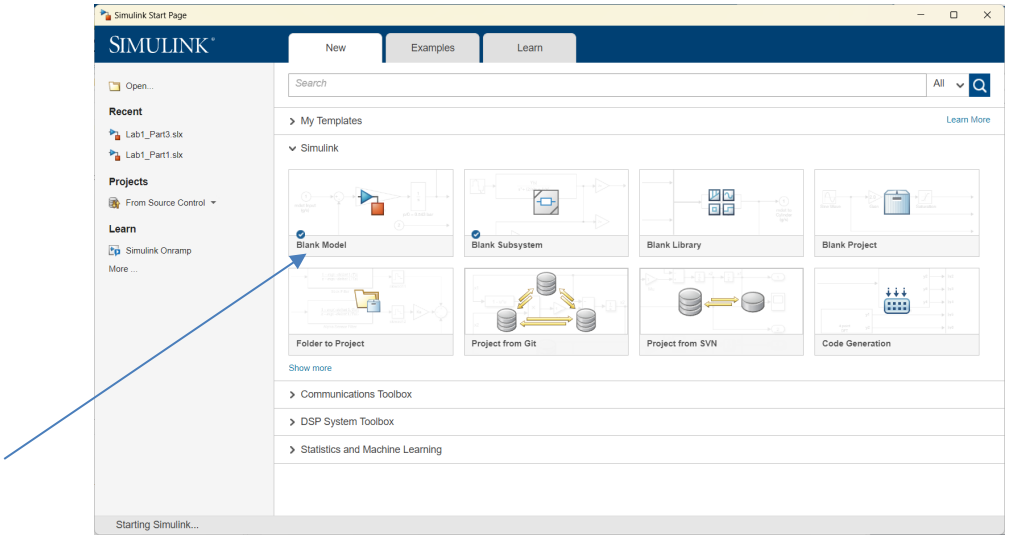
To start a Simulink session, you'd need to run Matlab program first. Then from Matlab command window, enter:

>> simulink

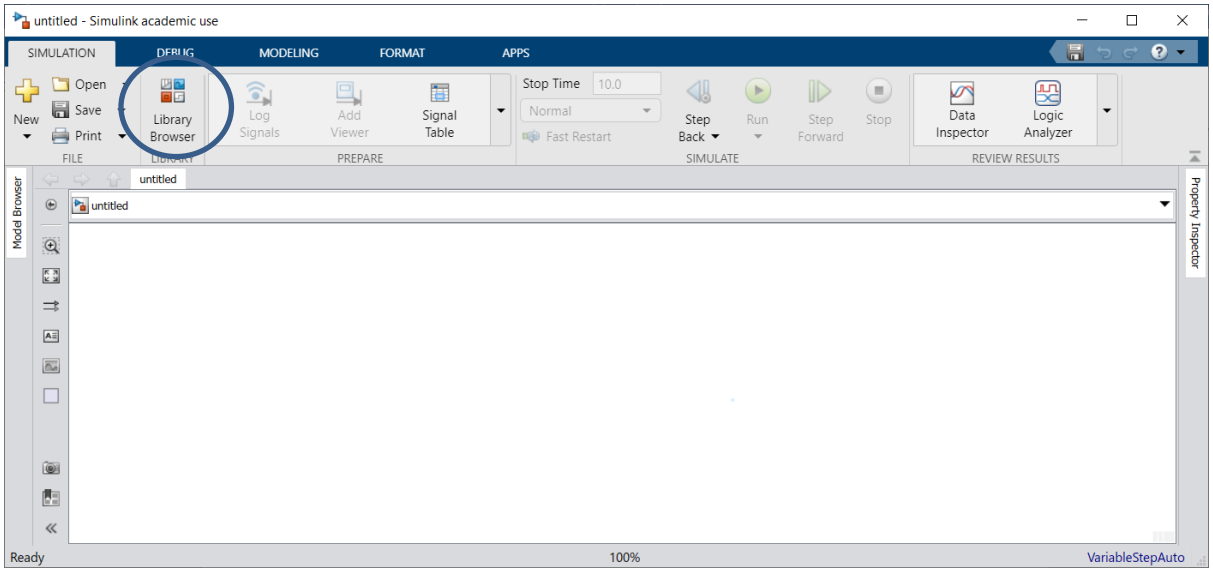
Alternately, you may click on the Simulink icon located on the toolbar as shown:



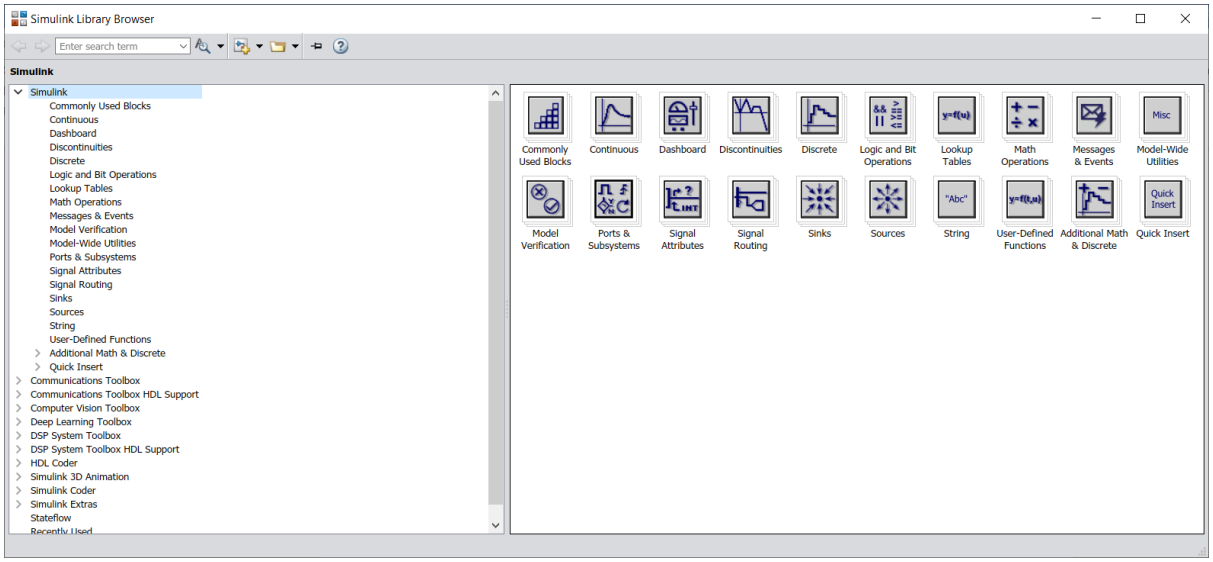
The Simulink start-up page will appear, giving you the option to open an existing model or start a new blank model. You can also choose a project folder at this stage.



To start a new model, click on the "Blank Model" icon, as shown in the screenshot above. A new window will open where you'll build your model. This same window is also used to run the simulation of your model. Below is a screenshot of a typical model window in use.



Now click on the “Library Browser”, this will bring up a single window, entitled Simulink Library Browser which is given on the next page. There are two major classes of items in Simulink: blocks and lines. Blocks are used to generate, modify, combine, output, and display signals. Lines are used to transfer signals from one block to another.



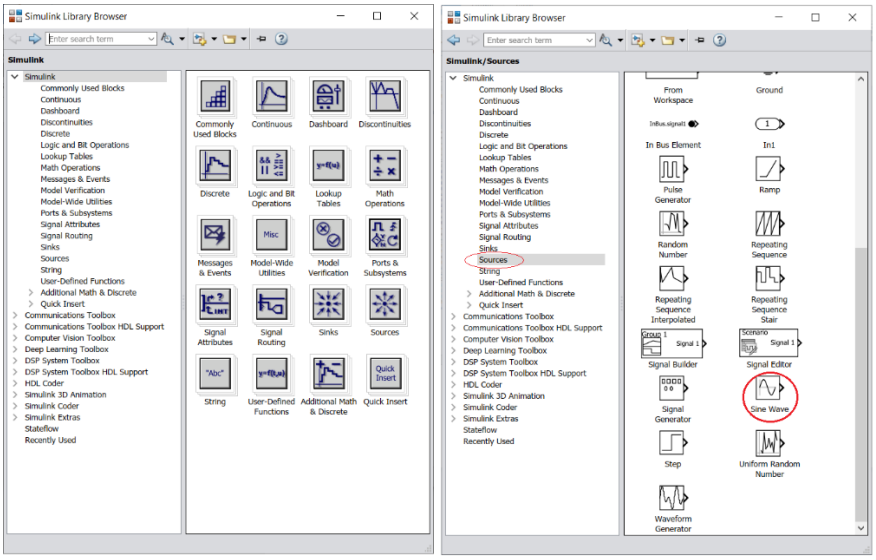
To familiarised with the structure and the environment of Simulink, you are encouraged to explore the toolboxes and scan their contents. You may not know what they are all about at first, but perhaps you could catch on the organisation of these toolboxes according to the category. A good way to learn Simulink (or any computer program in general) is to practice and explore.

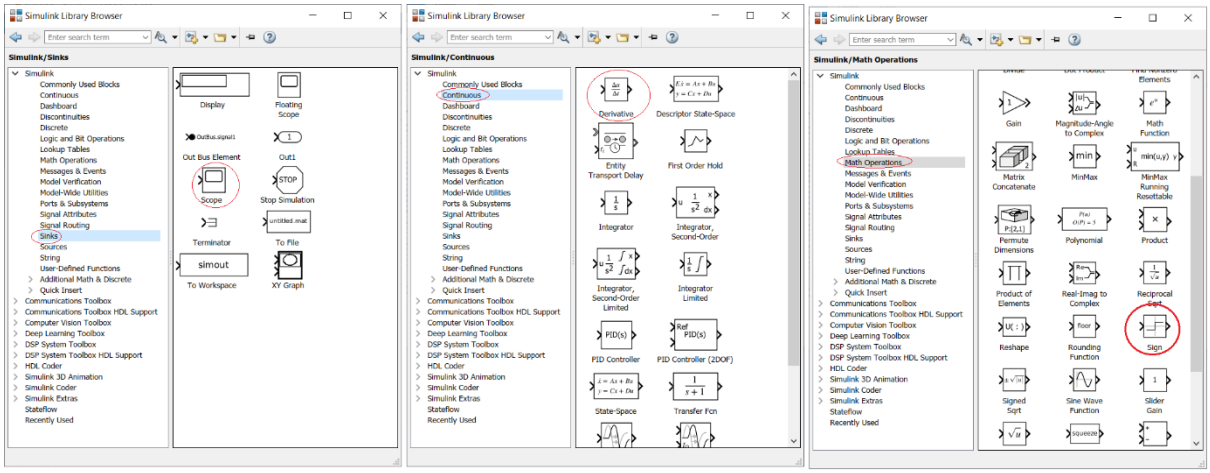
Let us create a simple model to understand the basic features of Simulink. Please follow the steps below to construct a simple model.

STEP 1: CREATING BLOCKS

To construct a model, one should first copy blocks from the Simulink Library Browser to the model window. To create a simple model in this example, we need four blocks:

- **Sine wave.** To generate an input signal for the model.
- **Derivative.** To differentiate the input signal.
- **Sign.** To generate a square wave from the sine wave.
- **Scope.** To display the signals (like an oscilloscope) in the model.





To add blocks to your model:

1. Select the **Sources** library in the Simulink Library Browser. The Simulink Library browser displays the Sources library.

2. Select the Sine Wave block in the Simulink Library Browser, then drag it to the model window. A copy of the Sine Wave block appears in the model window.

3. Select the **Sinks** library in the Simulink Library Browser.

4. Select the Scope block from the Sinks library, then drag it to the model window. A Scope block appears in the model window.

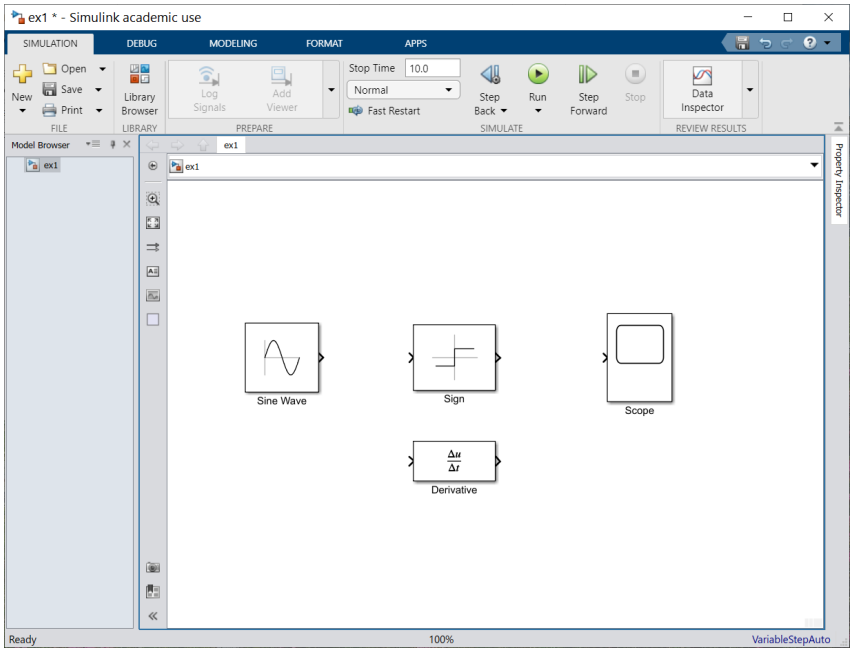
5. Select the **Continuous** library in the Simulink Library Browser.

6. Select the Derivative block from the Continuous library, then drag it to the model window. A Derivative block appears in the model window.

7. Select the **Math Operations** library in the Simulink Library Browser.

8. Select the Sign block from the Math Operations library, then drag it to the model window. A Sign block appears in the model window.

The figure below displays all the blocks in the model window.



Connecting Blocks in the Model Window

After blocks are added to the model window, we need to connect them to represent the signal connections within the model. Notice that each block has angle brackets on one or both sides. These angle brackets represent input and output ports.

- The > symbol pointing into a block is an input port.
- The > symbol pointing out of a block is an output port.

The blocks in the model are connected by drawing lines between output ports and input ports. To draw a line between two blocks:

1. Position the mouse pointer over the output port of the Sine Wave block. Note that the pointer turns into crosshairs (+) while over the port.
2. Click and drag a line from the output port to the input port of the Sign block. Note that the line is dashed while you hold the mouse button down and that the pointer turns into double crosshairs as it approaches the input port of the Sign block.
3. Release the mouse button over the input port. If the connection is successful, a solid line appears connecting the blocks with the arrow indicating the direction of signal flow. Otherwise, the line is red and dashed.
4. To weld, a connection to an existing line (“branch line”), position the mouse pointer on the line between the Sine Wave and the Sign block. Press and hold the Ctrl key, then drag a line to the Derivative block’s input port. The software draws a line between the starting point and the input port of the Derivative block.

Defining the Block Parameters

Next we specify the parameters of each block using the following steps:

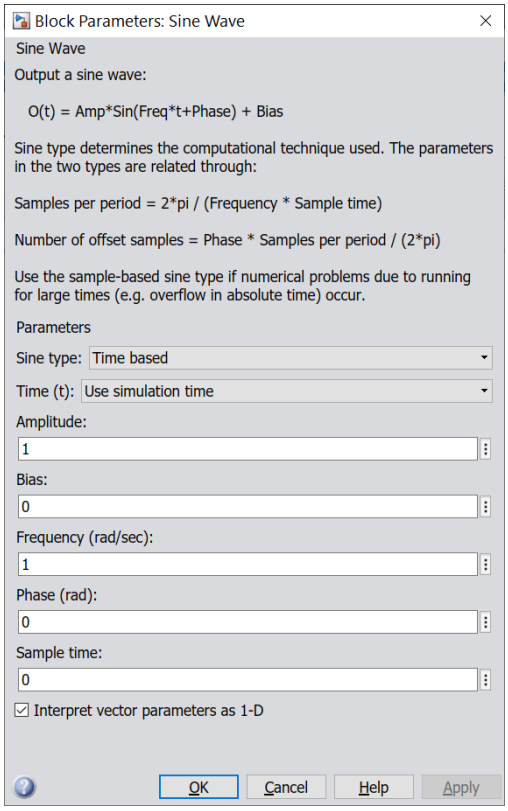
1. Double-click the block. This brings up the block Parameters dialog box.

2. Replace each relevant field with a proper value or selection.

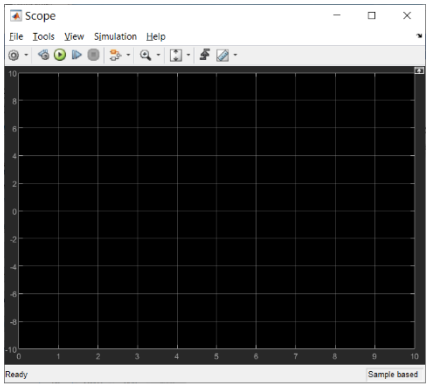
3. Click the OK button.

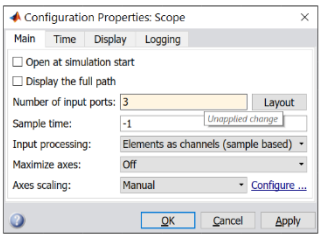
If we double-click the Sine Wave block, the dialog box appears as shown in figure below. The

Amplitude, Frequency, Phase, and Sample time parameters of the Sine Wave source are selected and then saved by clicking the **OK** button. Other blocks can keep their default parameter values.

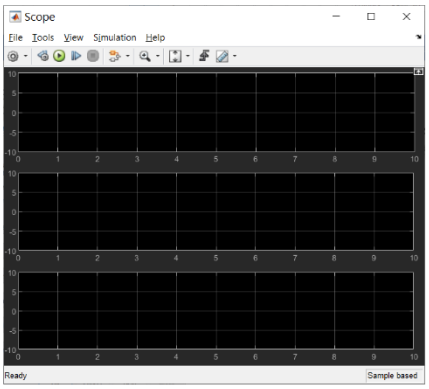


The Scope block provides a temporal display of multiple input waveforms. To select the number of input waveforms to be displayed, double-click the Scope block and change the parameters as shown in the figure below.



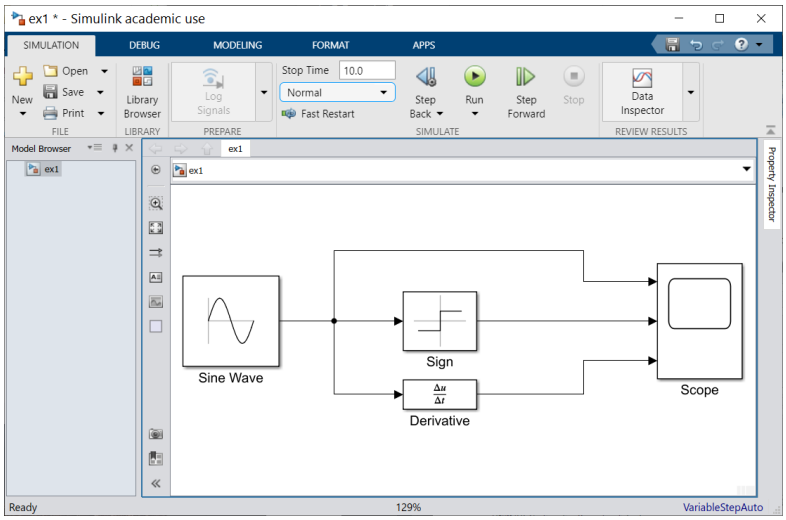


View -> Conifiguration Properties



View -> Layout

Link the blocks as shown below



Setting Simulation Options

Before running the simulation of the sample model, we can set simulation options such as the start and stop time, and the type of solver that Simulink software uses to solve the model at each time step. These options can be specified using the Configuration Parameters dialog box. To specify the simulation options for the sample model:

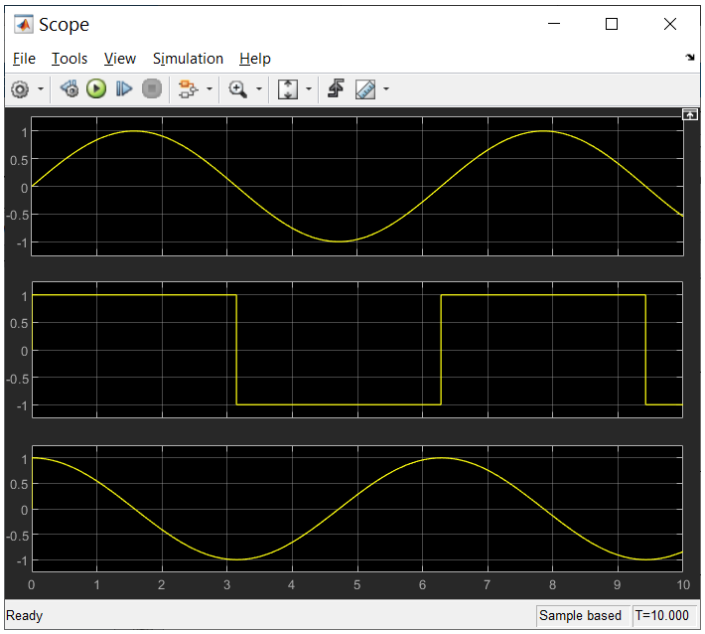
1. Select Simulation Parameters in the model window.
2. Enter 10 in the Stop time field.
3. Click OK.

Run the Simulation

To run the simulation:

1. Select Simulation + Run in the model window.
2. Double-click the Scope block in the model window. The Scope window displays the

simulation results



Saving the Model

After the model is completed, it can be saved as follows:

1. Select Simulation+ Save As in the model window.

2. Specify the location to which you want to save the model.

3. Enter the name in the File name field.

4. Click Save.